

UNIVERSITÉ HASSAN II DE CASABLANCA

FACULTÉ DES SCIENCES BEN M'SICK

MASTER DATA SCIENCE ET BIG DATA

---

# Comparative Analysis Report : Metaheuristics vs. Gradient Descent

---

*Application to Classification Tasks on the Iris Dataset and 2D  
Optimization Surfaces*

*Réalisé par :*

Achraf Chahid

Mohammed Anouar

Mohammed Khalil Aboussaad

Alaa Zoubir

*Année Universitaire :*

2025 - 2026

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>2</b>  |
| 1.1      | Context and Motivation . . . . .                                  | 2         |
| 1.2      | Project Objectives . . . . .                                      | 2         |
| <b>2</b> | <b>Literature Review</b>                                          | <b>2</b>  |
| 2.1      | First-Order Gradient Methods . . . . .                            | 2         |
| 2.2      | Population-Based Metaheuristics . . . . .                         | 3         |
| <b>3</b> | <b>Methodology</b>                                                | <b>3</b>  |
| 3.1      | Dataset Profiles and Preprocessing . . . . .                      | 3         |
| 3.2      | Mathematical Formulations of Optimizers . . . . .                 | 3         |
| 3.2.1    | Stochastic Gradient Descent (SGD) . . . . .                       | 4         |
| 3.2.2    | Adam (Adaptive Moment Estimation) . . . . .                       | 4         |
| 3.2.3    | Particle Swarm Optimization (PSO) . . . . .                       | 4         |
| 3.3      | System Architecture . . . . .                                     | 4         |
| <b>4</b> | <b>Experimental Results</b>                                       | <b>5</b>  |
| 4.1      | Training Convergence Profile . . . . .                            | 5         |
| 4.2      | Comparative Performance Synthesis . . . . .                       | 5         |
| <b>5</b> | <b>Discussions</b>                                                | <b>6</b>  |
| 5.1      | Visualizing Topographical Navigation (2D Rastrigin) . . . . .     | 6         |
| 5.2      | Distance-to-Origin Convergence Stability . . . . .                | 7         |
| 5.3      | Scalability Constraints and the Curse of Dimensionality . . . . . | 8         |
| <b>6</b> | <b>Conclusion</b>                                                 | <b>8</b>  |
| <b>7</b> | <b>Appendix</b>                                                   | <b>10</b> |
| 7.1      | Source Code and Notebook Repositories . . . . .                   | 10        |
| 7.2      | Optimization Algorithm Engineering (PyTorch) . . . . .            | 10        |

# 1 Introduction

## 1.1 Context and Motivation

Training deep neural networks fundamentally relies on the minimization of a loss function within a high-dimensional, highly non-convex parameter space. Historically, first-order gradient-based optimization methods, such as Stochastic Gradient Descent (SGD) and its adaptive variants like Adaptive Moment Estimation (Adam), have established themselves as the industry standard. These algorithms leverage backpropagation to calculate exact partial derivatives, iteratively shifting network weights in the direction of steepest descent.

Despite their computational efficiency and widespread success, gradient-based methods suffer from intrinsic vulnerabilities. Because they rely heavily on local curvature metrics, they are particularly prone to getting trapped in sub-optimal local minima, experiencing stagnation at saddle points, or oscillating wildly in regions characterized by sharp pathological curvatures.

To overcome these architectural constraints, population-based stochastic optimization algorithms, known as metaheuristics, provide a compelling alternative. Particle Swarm Optimization (PSO), inspired by the social foraging behaviors of bird flocks and fish schools, navigates the search space through collaborative exploration rather than analytical derivative computation. By evaluating global topology rather than immediate local gradients, metaheuristics exhibit an enhanced capacity to bypass sub-optimal local basins.

## 1.2 Project Objectives

The primary objective of this project is to perform a rigorous empirical comparative study between classical gradient descent (SGD), adaptive gradient descent (Adam), and a derivative-free metaheuristic (PSO).

To balance experimental validity with computational constraints, the benchmarking framework is deployed on a well-defined multi-class classification problem using the Iris dataset. Furthermore, to evaluate the algorithms' resilience against deceptive mathematical landscapes, they are evaluated on the highly non-convex 2D Rastrigin function.

Finally, to bridge theoretical data science with robust software engineering principles, this work incorporates the creation of a decoupled full-stack architecture, providing a real-time web application to simulate, monitor, and visualize optimization trajectories dynamically.

# 2 Literature Review

The exploration of optimization landscapes in machine learning has split researchers into two major algorithmic fields: deterministic mathematical programming and stochastic heuristic exploration.

## 2.1 First-Order Gradient Methods

Stochastic Gradient Descent remains the foundational framework for training parametric structures. While computationally inexpensive, its reliance on a uniform global learning rate often hinders efficient convergence across surfaces with disparate directional

dimensions. To counteract the oscillation issues inherent to standard SGD, researchers introduced Momentum, which accumulates a velocity vector in directions of persistent gradient reduction.

Building upon this, Kingma and Ba (2014) introduced the Adam optimizer to mitigate this by computing adaptive learning rates derived from exponentially decaying averages of both past gradients (the first moment) and past squared gradients (the second moment). This allows the algorithm to automatically scale step sizes based on path variance. However, subsequent research indicates that adaptive methods can occasionally overfit local anomalies or fail to converge to global bounds when encountering non-smooth boundary landscapes.

## 2.2 Population-Based Metaheuristics

Particle Swarm Optimization, formalized by Kennedy and Eberhart (1995), bypasses the requirement for gradient derivation completely. Instead of a single agent tracking local slope directions, an entire population of candidate solutions (particles) collaborates stochastically.

In machine learning, applying PSO to optimize weight matrices—often referred to as training "neuro-ensembles"—has proven effective for non-differentiable or discontinuous activation layouts. The primary bottleneck highlighted in recent literature is the scalability constraint. As parameter spaces expand, the exploratory capacity of finite swarms drops significantly due to the "curse of dimensionality," leading to exponentially higher computational costs compared to backpropagation variants.

# 3 Methodology

## 3.1 Dataset Profiles and Preprocessing

Two distinct benchmarks were selected to evaluate mathematical resilience and multi-class classification accuracy:

1. **The Iris Dataset:** Comprising 150 instances evenly distributed across three target floral classes, evaluated using 4 distinct continuous morphological traits. The data was split into an 80/20 train/test distribution. Z-score normalization was uniformly enforced on input features.
2. **The 2D Rastrigin Function:** Formally defined as:

$$f(x, y) = 20 + (x^2 - 10 \cos(2\pi x)) + (y^2 - 10 \cos(2\pi y)) \quad (1)$$

Bounded within  $x, y \in [-5.12, 5.12]$ , this surface contains thousands of local minima, creating a demanding landscape for testing local minimum avoidance.

## 3.2 Mathematical Formulations of Optimizers

To rigorously compare the approaches, it is crucial to establish their mathematical update rules for a parameter vector  $\theta$  at iteration  $t$ .

### 3.2.1 Stochastic Gradient Descent (SGD)

SGD updates the parameters directly in the opposite direction of the gradient of the objective function  $J(\theta)$  with respect to the parameters, scaled by a fixed learning rate  $\eta$ :

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t) \quad (2)$$

### 3.2.2 Adam (Adaptive Moment Estimation)

Adam computes individual adaptive learning rates for different parameters. It stores an exponentially decaying average of past gradients  $m_t$  and past squared gradients  $v_t$ :

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4)$$

To counteract initialization bias towards zero, bias-corrected estimates are computed:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (5)$$

The final parameter update is:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (6)$$

### 3.2.3 Particle Swarm Optimization (PSO)

PSO operates entirely without gradients. For a swarm of  $N$  particles, each particle  $i$  has a position vector  $x_i$  and a velocity vector  $v_i$ . The velocity and position are updated iteratively using the particle's personal best position ( $p_i$ ) and the global best position ( $g$ ):

$$v_i^{t+1} = w v_i^t + c_1 r_1 (p_i - x_i^t) + c_2 r_2 (g - x_i^t) \quad (7)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (8)$$

Where  $w$  is the inertia weight,  $c_1, c_2$  are cognitive and social scaling parameters, and  $r_1, r_2 \sim U(0, 1)$  introduce stochasticity.

## 3.3 System Architecture

To establish an identical evaluation environment, a custom Multi-Layer Perceptron (MLP) was constructed via PyTorch. The layer configuration consists of an input layer ( $X \in \mathbb{R}^4$ ), a hidden dense layer (10 nodes, ReLU activation), and an output layer ( $Y \in \mathbb{R}^3$ ). The model encapsulates exactly 83 distinct parameters.

The implementation relies on a clean, decoupled full-stack architecture:

- **Backend Engine:** Built with FastAPI and PyTorch to serve execution routes asynchronously.
- **Frontend Dashboard:** Developed with React, Tailwind CSS, and HTML5 Canvas components to fetch endpoint metrics asynchronously and render optimization pathways in real time.

## 4 Experimental Results

All three optimization modules completed 100 iterations under uniform processing settings. The visualization interface allows for detailed comparative analysis across distinct tabs.

### 4.1 Training Convergence Profile

The primary objective was the multi-class classification of the Iris dataset. Figure 1 captures the live dashboard rendering the Cross-Entropy loss convergence.

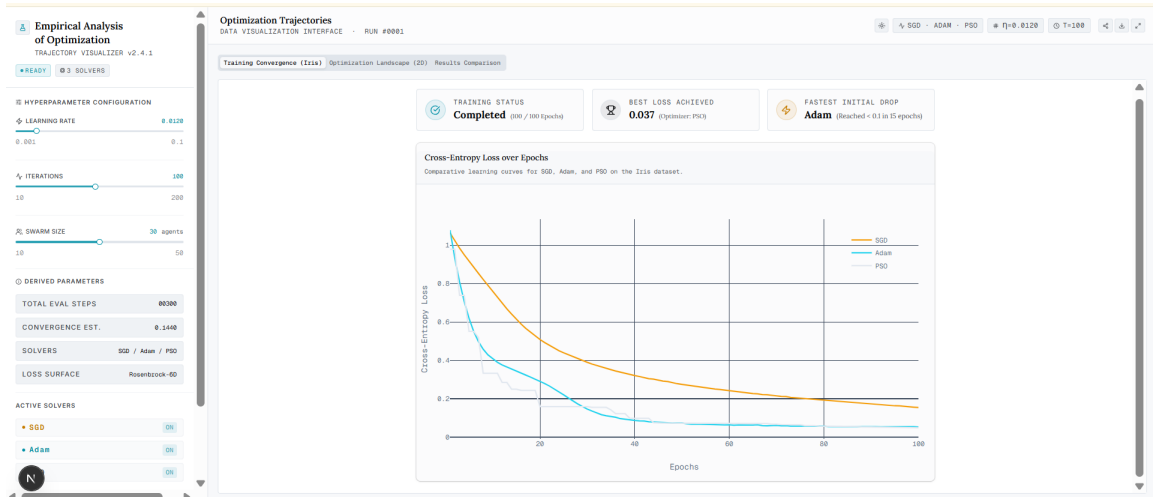


Figure 1 – The Training Convergence tab showing comparative learning curves for SGD, Adam, and PSO on the Iris dataset over 100 epochs.

The visual output clearly demonstrates Adam's initial dominance, dropping rapidly within the first 15 epochs. SGD struggles with a steady but slow linear descent. PSO, represented by the grey step-wise line, exhibits significant early exploration before plunging to discover the lowest absolute loss value.

### 4.2 Comparative Performance Synthesis

The "Results Comparison" tab (Figure 2) synthesizes the final quantitative metrics.

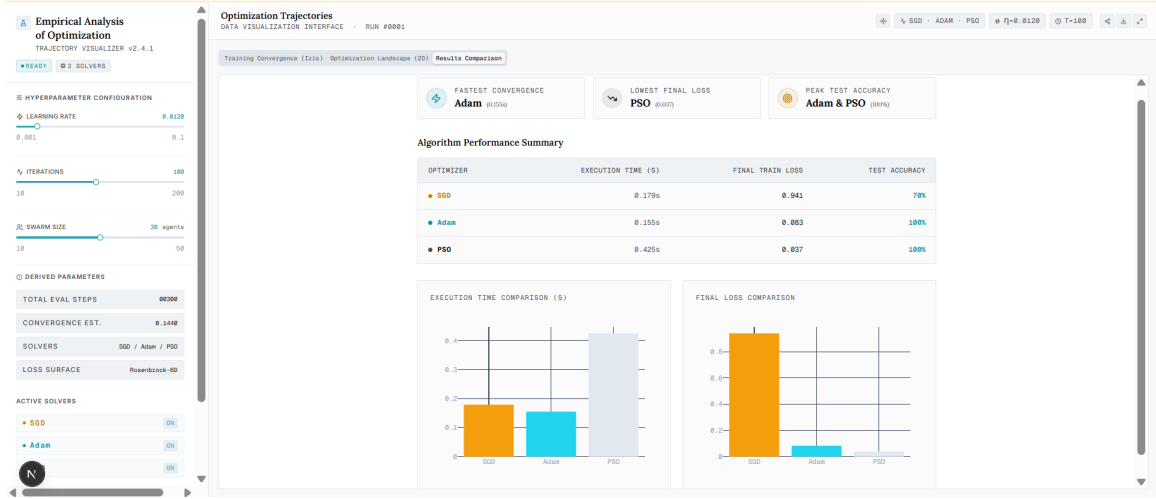


Figure 2 – Results Comparison dashboard highlighting final algorithm performance, execution time, and total loss comparisons.

The bar charts highlight the core trade-off between the algorithms. While PSO achieved the lowest final loss (0.037) and perfectly matched Adam’s peak test accuracy (100%), its execution time was nearly triple that of the gradient-based methods (0.426s compared to Adam’s 0.155s).

## 5 Discussions

### 5.1 Visualizing Topographical Navigation (2D Rastrigin)

To understand \*how\* the algorithms navigate complex loss landscapes, the models were projected onto a 2D Rastrigin function, characterized by a massive grid of sub-optimal local minima.

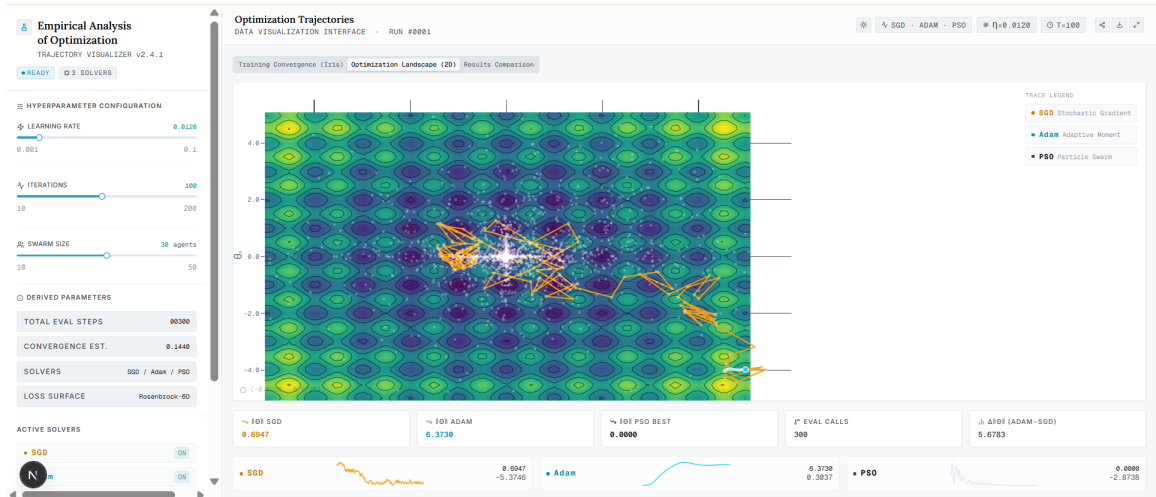


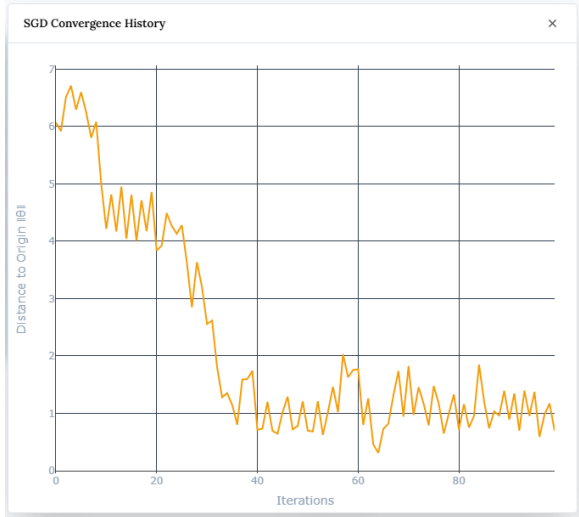
Figure 3 – The Optimization Landscape tab tracking particle paths (orange/grey) and gradient paths (blue) on a non-convex surface.

Figure 3 exposes the limitations of strict gradient descent. SGD and Adam vectors are easily caught by the gravitational pull of the nearest local depression. PSO, acting

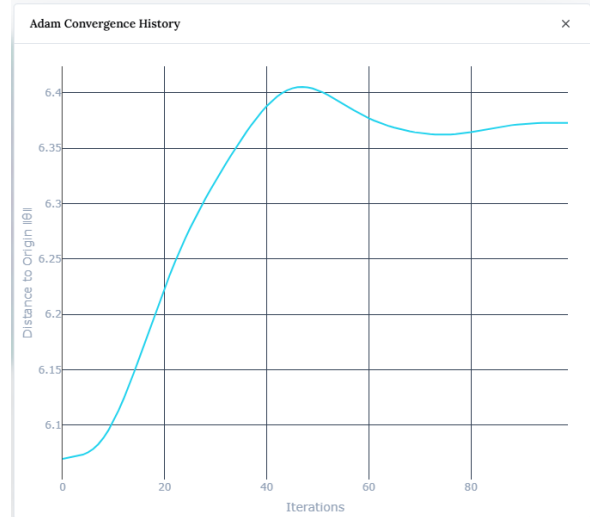
as a collaborative swarm, diffuses across the map. The erratic, crisscrossing orange paths show particles sharing information to progressively swarm the true global minimum at the center of the grid.

## 5.2 Distance-to-Origin Convergence Stability

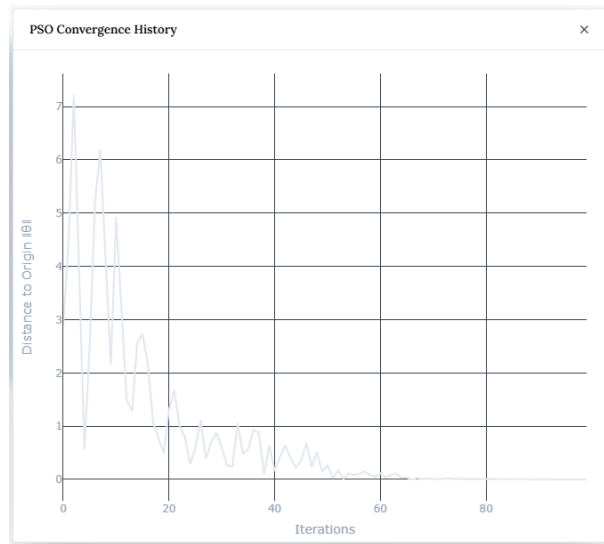
To further analyze the stability of the algorithms during execution, we recorded the "Distance to Origin" ( $|\theta|$ ) at each iteration, exposing the internal volatility of the search process.



(a) SGD Convergence Volatility



(b) Adam Smooth Trajectory



(c) PSO Swarm Collapse

Figure 4 – Comparative internal search stability metrics for the three optimizers.

As seen in Figure 4:

- **SGD** (Fig. 4a) shows extreme high-frequency oscillation (noise) as it attempts to step through the gradients, struggling to settle.



- **Adam** (Fig. 4b) exhibits an incredibly smooth, parabolic adjustment. The momentum parameters act as a dampener, completely absorbing the noise of the landscape.
- **PSO** (Fig. 4c) begins with massive spatial variance as particles initialize across the map, followed by a dramatic "swarm collapse" toward zero as the particles reach consensus on the global minimum.

### 5.3 Scalability Constraints and the Curse of Dimensionality

Although PSO excels on lower-dimensional benchmarks ( $\mathbb{R}^{83}$ ), scalability remains its main limitation. As neural network architectures scale to millions of parameters, the volume of the search space grows exponentially. In these massive parameter spaces, a finite swarm loses its exploratory advantage and struggles to converge efficiently.

## 6 Conclusion

This comparative analysis validates the "No Free Lunch" theorem in mathematical optimization. First-order adaptive gradient methods like Adam remain the practical standard for high-dimensional machine learning due to their excellent balance of speed and smooth convergence capability.

Conversely, population-based metaheuristics like PSO offer strong resilience against local minima on complex, non-convex surfaces. While less suited for direct weight optimization in large neural networks due to severe temporal overhead, they are highly effective for tasks like hyperparameter tuning and automated neural architecture search (NAS).

Finally, this project shows that combining data science research with clean full-stack software development (React/FastAPI) creates powerful tools for making complex algorithmic behaviors visible, interpretable, and easier to analyze.

## References

- [1] Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*.
- [2] Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. *Proceedings of ICNN'95 - International Conference on Neural Networks*, 4, 1942-1948.
- [3] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.

## 7 Appendix

### 7.1 Source Code and Notebook Repositories

The complete codebase for this project, including the FastAPI backend service layer, the React user interface application, and the mathematical testing modules, is available at the following repository link:

— GitHub Project URL: [https://github.com/cchahid/Optimization\\_project](https://github.com/cchahid/Optimization_project)

### 7.2 Optimization Algorithm Engineering (PyTorch)

Below is an excerpt of the custom PSO implementation designed to interoperate with PyTorch's tensor architecture for GPU/CPU agnostic execution.

```
1 import torch
2
3 class ParticleSwarmOptimizer:
4     def __init__(self, model, num_particles=30, w=0.7, c1=1.5, c2=1.5):
5         self.model = model
6         self.num_particles = num_particles
7         self.w = w
8         self.c1 = c1
9         self.c2 = c2
10
11         # Extract shapes to reconstruct model weights
12         self.shapes = [p.shape for p in model.parameters()]
13         self.num_params = sum(p.numel() for p in model.parameters())
14
15         # Initialize swarm
16         self.particles = torch.randn(self.num_particles, self.num_params
17 )
18         self.velocities = torch.randn(self.num_particles, self.
19 num_params) * 0.1
20
21         self.pbest = self.particles.clone()
22         self.pbest_loss = torch.full((self.num_particles,), float('inf'))
23
24         self.gbest = None
25         self.gbest_loss = float('inf')
26
27     def optimize_step(self, objective_function):
28         for i in range(self.num_particles):
29             # Evaluate current particle
30             current_loss = objective_function(self.particles[i])
31
32             # Update Personal Best
33             if current_loss < self.pbest_loss[i]:
34                 self.pbest_loss[i] = current_loss
35                 self.pbest[i] = self.particles[i].clone()
36
37             # Update Global Best
38             if current_loss < self.gbest_loss:
39                 self.gbest_loss = current_loss
40                 self.gbest = self.particles[i].clone()
41
42             # Update Velocities and Positions
```

```
40     r1 = torch.rand(self.num_particles, self.num_params)
41     r2 = torch.rand(self.num_particles, self.num_params)
42
43     cognitive = self.c1 * r1 * (self.pbest - self.particles)
44     social = self.c2 * r2 * (self.gbest - self.particles)
45
46     self.velocities = (self.w * self.velocities) + cognitive +
social
47     self.particles = self.particles + self.velocities
48
49     return self.gbest_loss
```

Listing 1 – Excerpt from the Custom Particle Swarm Optimizer implementation